

BIG DATA E INGENIERÍA DE DATOS – PARCIAL III

(Máximo 2 estudiantes)

ShopStream es una empresa ficticia de e-commerce cuyo equipo de producto necesita entender cómo los usuarios interactúan con su sitio web: qué páginas visitan, cuánto tiempo permanecen en cada una, en qué punto abandonan el flujo de compra y qué productos reciben más clics sin llegar a ser comprados. Esta información es capturada por el servidor web como logs de comportamiento y constituye la fuente de datos principal del pipeline.

Cada noche, a las 00:00 UTC, el servidor web de ShopStream consolida todos los eventos de interacción del día en un único archivo de log y lo deposita automáticamente en Amazon S3. A partir de ese momento se activa el pipeline de datos batch.

Se deberá construir un pipeline completo en AWS que procese estos logs de comportamiento: validarlos con Lambda, transformarlos y calcular métricas de experiencia de usuario con PySpark sobre EMR, catalogarlos y orquestarlos con Glue, y finalmente exponer indicadores clave vía una API Lambda conectada a RDS.

Punto 1 — Diseño y Generación de Datos

Se debe diseñar el modelo de datos de ShopStream: esquema de eventos, catálogo de productos, usuarios y transacciones. Luego generar un dataset sintético realista.

- Diseñar el esquema de eventos (JSON/CSV) con al menos 5 entidades relacionadas
- Escribir un script Python que genere $\geq 500,000$ registros con distribuciones realistas a diario
- Subir los datos en particiones (año/mes/día) a S3 bucket raw

formatos de eventos:

- page_view: usuario visita una página (user_id, session_id, page_url, page_type, timestamp, time_on_page_seconds, referrer, device_type, country)
- click: usuario hace clic en un elemento (user_id, session_id, element_id, element_type, page_url, timestamp, x_position, y_position)
- search: usuario realiza una búsqueda (user_id, session_id, query, results_count, timestamp)
- product_view: usuario ve la página de un producto (user_id, session_id, product_id, category, price, timestamp, time_on_page_seconds)
- cart_event: usuario agrega o elimina un producto del carrito (user_id, session_id, product_id, action: add/remove, timestamp)

Punto 2 — Ingesta y Trigger Automático

Configurar un mecanismo de disparo automático que reaccione a nuevos archivos en S3 e inicie el pipeline de procesamiento.

- Crear una función Lambda que se active con eventos S3 PutObject
- Implementar validación del esquema (tipos, campos obligatorios, rangos) en la función
- Mover archivos inválidos a una carpeta quarantine/ con metadata del error
- Registrar métricas de ingesta en CloudWatch (archivos procesados, errores, tamaño)

Punto 3 — Transformación Distribuida con PySpark

- Configurar un cluster EMR con las versiones correctas de Spark y Python
- Implementar limpieza de datos: deduplicación, imputación de nulos, normalización
- Calcular métricas de comportamiento agregadas por día:
 - Páginas con mayor tiempo de permanencia promedio (top 20 por page_url)
 - Tasa de rebote por tipo de página (home, categoría, producto, carrito): porcentaje de sesiones con un único page_view
 - Embudo de conversión: cuántos usuarios pasan de page_view a product_view, de product_view a cart_event, y de cart_event a compra completada
 - Productos más vistos vs productos más agregados al carrito: identificar productos con alta vista pero baja conversión (candidatos a mejora de precio o descripción)
 - Rutas de navegación más frecuentes: las 10 secuencias de páginas más comunes usando el campo de secuencia de sesión
 - Tiempo promedio en página por tipo de dispositivo (mobile vs desktop vs tablet) y por país
- Detección de anomalías usando estadística descriptiva (z-score, IQR) .
- Exportar resultados a S3 en formato Parquet particionado
- El pipeline debe funcionar tanto como notebook como script para ejecutar con spark-submit

Punto 4. Glue

- Crear una base de datos postgres tipo dataware house para almacenar los datos de analítica.
 - Construir uno de los jobs ETL visualmente en Glue Studio (carga datos de s3 hacia RDS) usando el editor de nodos gráfico
 - Agregar un nodo de Data Quality en el grafo de Glue Studio para validar que los registros de salida cumplen reglas mínimas (por ejemplo, formato json, formato de fechas correctas)
 - Configurar Triggers condicionales: el Job de carga a RDS solo debe ejecutarse si el Job ETL terminó con estado SUCCEEDED; si falla, enviar alerta
 - Agregar un trigger programado (schedule) para ejecutar el workflow completo diariamente a las 2:00 AM UTC

API de Consultas y Presentación

Exponer los datos procesados mediante una API REST funcional en lambda que permita consultar métricas de negocio en tiempo real del sistema de datawarehouse.(Usar zappa)

- Crear mínimo 3 endpoints Lambda conectados vía API Gateway:

- GET /pages/top?metric={bounce_rate|time_on_page}&date={date}&limit={n} — retorna las páginas con mayor tasa de rebote o tiempo de permanencia para una fecha dada
- GET /sessions/summary?country={country}&device={device}&date={date} — resumen de sesiones filtrado por país y dispositivo
- GET /anomalies?date={date} — lista de sesiones anómalas detectadas ese día con su z_score y tipo de anomalía

** Se debe entregar en github con pruebas unitarias y despliegue continuo.